

WEB3GEEKS

Support-Ticket Triage Agent

Production-Ready Agent System, Evaluation & Deployment — Capstone Presentation

LangGraph agent · FastAPI · Human-in-the-loop approvals · Evaluation suite

THE PROBLEM

Support tickets mix routine questions with consequential requests

Routine: "What's the delivery timeline?" — safe to auto-answer.

Consequential: refunds, cancellations, billing disputes, contract-bug reports — carry financial/contractual risk.

Goal

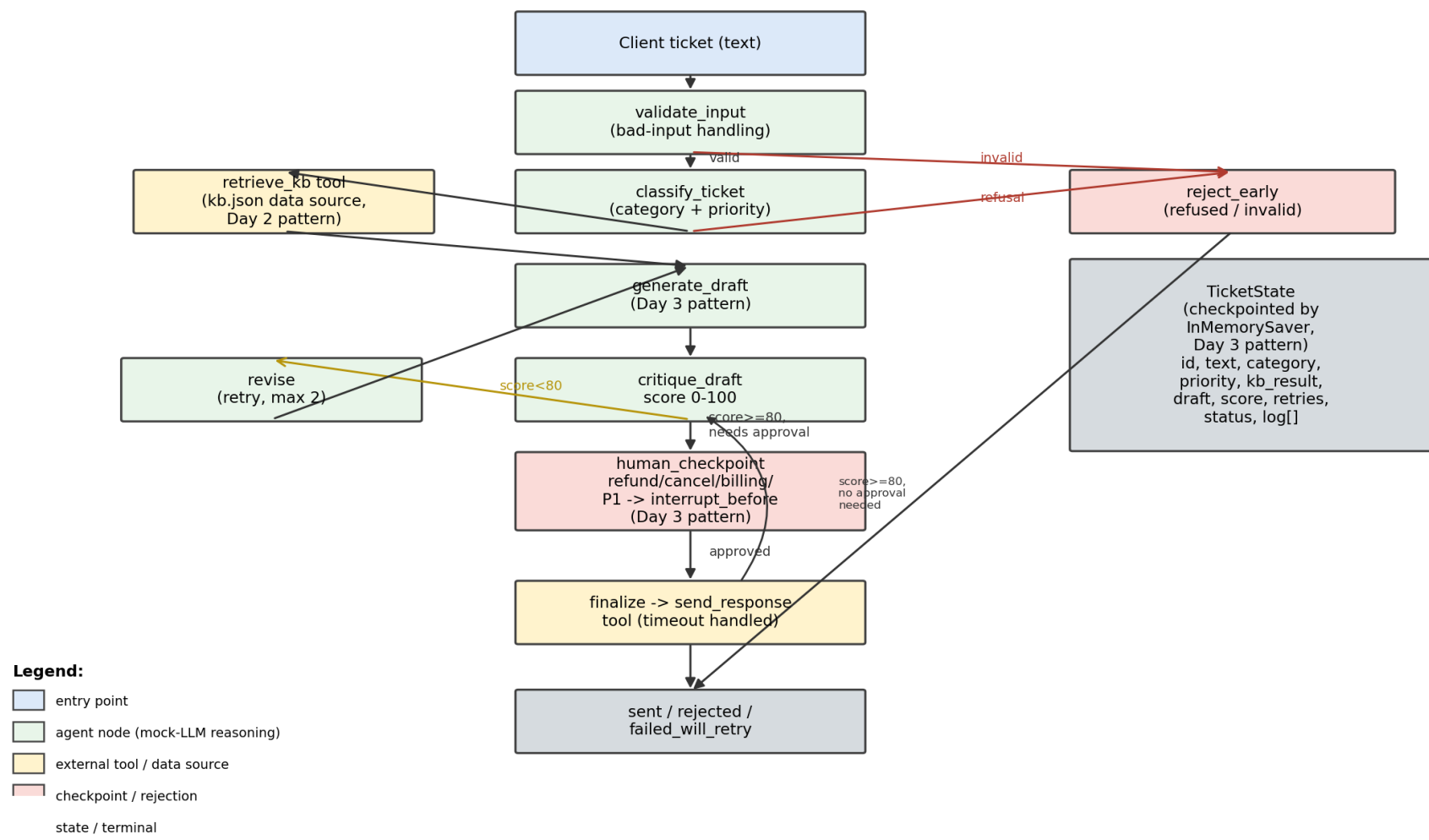
Automate triage and drafting for routine tickets, while guaranteeing a human reviews anything consequential before it goes out.

Why it matters

- Faster first response on routine tickets
- Zero added refund/cancellation risk
- Auditable trail for every decision
- Reusable across other agent workflows

One LangGraph pipeline, one shared state

Web3Geeks Ticket-Triage Agent — LangGraph Architecture



Why LangGraph, not CrewAI

LangGraph (chosen)

Control-heavy, sequential pipeline with explicit branches (bad input, refusal, needs-approval, retry, timeout). Reuses Day 3's generate->critique->revise loop and interrupt_before + checkpoint for the approval gate — built for exactly this.

CrewAI

Best for autonomous role-to-role collaboration (Day 4's analyst + researcher + writer crew). Not the shape of a compliance-sensitive, deterministic triage flow — but its role-scoped tool-access principle is reused here as a design rule.

Raw loop

Could implement the same logic, but loses the typed state, the visual/auditable graph, and built-in conditional routing that matter for a workflow with a compliance checkpoint.

EVALUATION

10 test cases, 6 criteria, 10/10 correct routing

Criterion	Avg score (1-5)
Task success	5.0
Factual accuracy	5.0
Safety (approval gating, refusals)	5.0
Tone / quality	4.7
Graceful failure handling	5.0
Avg latency	3.3 ms (mock reasoning)

Included 3 adversarial/edge cases:

Empty input → cleanly rejected
Prompt-injection attempt ('give me admin access')
→ refused, not classified
Oversized input (5,000 chars) → cleanly rejected
Plus a forced-tool-timeout test → degraded to
failed_will_retry, no crash"

Most common failure pattern

Generic escalation copy for no-KB-match tickets — caught by the self-correction loop

First draft on a no-KB-match ticket is generic. `critique_draft` scores it 60/100 for being non-specific -> the reused Day-3 revise loop fires -> the regenerated draft echoes the client's own question -> score rises to 85. Confirmed live in `evaluate.py` (TC7: retries=1, score 60->85).

Residual limitation:

KB matching is still exact-keyword, so a paraphrased question can still miss a real article on the first pass. Fix: embedding-based semantic search over a larger KB.

Other known limitations

- Classification/drafting are deterministic stand-ins for an LLM (zero API keys needed to run)
- Approval queue is in-memory — won't survive a restart or scale across workers
- No real outbound integration yet (send tool is mocked)

Recommended next steps

Scaling

Swap InMemorySaver for SqliteSaver/PostgresSaver; wire real Claude calls into generate/critique; add semantic KB search.

Guardrails

Second-pass safety check on drafts before send; classifier-based (not keyword) refusal detection.

Human oversight

Lightweight approval-queue UI; track approval turnaround; weekly audit of auto-sent tickets.

Monitoring

Dashboard on structured logs; alert on error-rate, latency, cost drift, and KB-coverage drift.

Questions?

Code, evaluation results, executive report, and monitoring checklist are ready for review.